



Univerzitet u Beogradu - Elektrotehnički fakultet

Katedra za računarsku tehniku i informatiku

## **DIPLOMSKI RAD**

# **PROCENA PERFORMANSI CIKLIČNE PROVERE REDUNDANSE U SAVREMENIM KOMUNIKACIONIM PROTOKOLIMA**

### **Kandidat**

Jelena Bakić, 2018/0135

### **Mentor**

dr Srđan Brkić, docent

Beograd, *februar* 2023. godine

## SADRŽAJ

|                                                         |    |
|---------------------------------------------------------|----|
| SADRŽAJ.....                                            | 1  |
| 1 UVOD .....                                            | 2  |
| 2 LINEARNI BLOK KODOVI .....                            | 3  |
| 2.1 Uvod.....                                           | 3  |
| 2.2 Matrični opis .....                                 | 5  |
| 2.3 Verovatnoća greške koja se ne može detektovati..... | 7  |
| 3 CIKLIČNI KODOVI .....                                 | 11 |
| 3.1 Uvod.....                                           | 11 |
| 3.2 Primeri .....                                       | 14 |
| 4 CIKLIČNA PROVERA REDUNDANSE (CRC) .....               | 17 |
| 4.1 Uvod.....                                           | 17 |
| 4.2 CRC postupak.....                                   | 18 |
| 4.3 Performanse CRC postupka .....                      | 19 |
| 5 ANALIZA SIMULACIJE U PROGRAMU .....                   | 22 |
| 5.1 Opis protokola korišćenih u simulaciji.....         | 22 |
| 5.1.1 Bluetooth.....                                    | 22 |
| 5.1.2. USB.....                                         | 22 |
| 5.1.3. WLAN.....                                        | 22 |
| 5.2 Opis simulacije .....                               | 23 |
| 6 ZAKLJUČAK.....                                        | 26 |
| 7 LITERATURA.....                                       | 27 |
| 8 DODATAK.....                                          | 28 |

## 1 UVOD

U današnje vreme glavni način prenosa informacija je digitalni prenos. Sve informacije predstavljaju se binarno. Kod digitalnog prenosa usled šuma može doći do greške pri prenosu pojedinačnih bita ili do greške na grupi bita. Zbog ovoga se uvodi zaštitno kodovanje. Zaštitno kodovanje daje mogućnost detekcije i korekcije greške pri prenosu informacija.

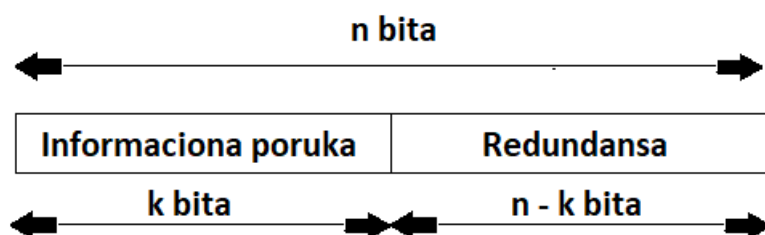
U ovom radu biće reč o zaštitnim kodovima, konkretno o Linearnim blok kodovima. Postoje vrste zaštitnih kodova koje pružaju mogućnost samo detekcije grešaka, ali se mogu kombinovati sa drugim vrstama zaštitnih kodova koje omogućavaju i korekciju grešaka.

Ciklični kodovi, potklasa linearnih kodova, biće opisani teorijski, kao i kroz primere, sa naglaskom na kodove za cikličnu proveru redundanse (CRC) koji imaju mogućnost samo da otkriju greške, ali ne i da ih koriguju.

Takođe, biće reč i o telekomunikacionim protokolima korišćenim u simulaciji u kojoj je opisana procena performansi ciklične provere redundanse. Program za simulaciju je izrađen na programskom jeziku C++.

## 2 LINEARNI BLOK KODOVI

Kodovi o kojima će biti reč u ovom poglavlju su linearni blok kodovi. U opštem slučaju, blok kod  $(n,k)$  može se zadati tabelom u kojoj su sa jedne strane blokovi od po  $k$  informacionih bita (izabranih od  $2^k$  mogućnosti), a sa druge strane odgovarajuće kodne reči dužine  $n$  (od  $2^n$  mogućih reči). Svaki blok podataka dužine  $k$  se obrađuje nezavisno od prethodnih blokova podataka. Blok kodovi funkcionišu tako što se na informacionu poruku dužine  $k$  bita, dodaje  $n-k$  redundantnih simbola, da bi se na prijemu mogla detektovati greška nastala pri prenosu informacione poruke kroz kanal (slika 2.1).



Slika 2.1 Informaciona poruka sa dodatkom redundanse

### 2.1 Uvod

**Linearni kod** je potprostor vektorskog prostora nad poljem Galoa  $GF(q)$ , konačnim poljem sa  $q$  simbola. U polju  $GF(q)$  je definisano sabiranje i množenje elemenata (komutativne operacije, prikazane u tabelama 1 i 2). Vektorski prostor  $V$  dimenzije  $n$  nad datim poljem čine sekvence od po  $n$  elemenata polja. Ako je broj elemenata u kodnoj listi prost broj ili stepen prostog broja, postoje linearni blok kodovi. Od posebnog značaja je što za svaki stepen broja 2 postoji linearni blok kod. Postoje i „nelinearni“ kodovi, kodovi sa bilo kojim brojem elemenata u kodnoj listi, ali ovakvi kodovi ne podležu prethodno opisanom matematičkom pristupu. Teorija opisana u nastavku važi za linearne blok kodove sa  $q$  različitih simbola, ali biće posmatrano polje sa parametrom  $q = 2$  ( $GF(2)$ ).

Prethodna definicija linearnog koda može se preformulisati. Linearni kod se dobija ako se od  $2^n$  kandidata za kodne reči izabere  $2^k$  kodnih reči koje čine potprostor posmatranog vektorskog prostora. Po Lagranžovoj teoremi, za svako  $n$  i  $k$  postoji linearni kod  $(n, k)$ , pošto je  $2^n$  deljivo sa  $2^k$  (teorema kaže da broj elemenata grupe mora biti deljiv brojem elemenata podgrupe, a vektori čine grupu u odnosu na sabiranje). Za podgrupu važi zatvorenost i postojanje neutralnog elementa, pa tako važi da zbir bilo koje dve kodne reči mora dati kodnu reč takođe, kao i da u linearnom kodu postoji kodna reč (00..0) kao neutralni element.

**Težina kodne reči** predstavlja broj njenih nenultih bitova. Kodna reč svih nula se može smatrati kao “rastojanje vektora od samog sebe”. **Hemingovo rastojanje** dve kodne reči je jednako težini njihove razlike. **Spektar kodnih rastojanja** (raspodela težina) je bitan za određivanje verovatnoće greške i pokazuje koliko u kodu ima kodnih reči određene težine. Spektar kodnih rastojanja u potpunosti opisuje neki linearni blok kod. On pokazuje broj reči određene Hemingove težine u datom kodu. Kodovi koji imaju isti spektar nazivaju se ekvivalentni kodovi.

Tabela 2.1 Sabiranje u polju GF(2)

| $x_1$ | $x_2$ | $x_1 + x_2$ |
|-------|-------|-------------|
| 0     | 0     | 0           |
| 0     | 1     | 1           |
| 1     | 0     | 1           |
| 1     | 1     | 0           |

Tabela 2.2 Množenje u polju GF(2)

| $x_1$ | $x_2$ | $x_1 * x_2$ |
|-------|-------|-------------|
| 0     | 0     | 0           |
| 0     | 1     | 0           |
| 1     | 0     | 0           |
| 1     | 1     | 1           |

### Primer 2.1: Težina kodnih reči i Hemingovo rastojanje

Kodna reč 1001100 ima težinu 3, a težina reči 1000000 je jednaka 1.

Hemingovo rastojanje  $d(1001100, 1000000) = 2$

Težina razlike  $1001100 - 1000000 = 0001100$  takođe je jednaka 2.

## 2.2 Matrični opis

Linearni kod može se opisati skupom linearno nezavisnih vektora (**bazom**) čijim se sabiranjem dobijaju svi vektori prostora. Za kod  $(n, k)$  važi da je dimenzija prostora jednaka  $n$ , a dimenzija potprostora je  $k$ , pa važi i to da je broj gore navedenih vektora jednak  $n$ , odnosno dimenziji prostora. Vektori se mogu predstaviti matricom dimenzija  $k \times n$ , koja se još naziva i **generišuća matrica**. Njen rang je  $k$  i ima linearno nezavisne vrste.

Množenjem informacione poruke  $\mathbf{i}$ , dužine  $k$  i generišuće matrice  $\mathbf{G}$ , dobija se kodna reč  $\mathbf{c}$ , dužine  $n$ .

$$\mathbf{i} = i_1 i_2 i_3 \dots i_k \quad (2.1)$$

$$\mathbf{G} = \begin{bmatrix} g_{11} & \cdots & g_{1n} \\ \vdots & \ddots & \vdots \\ g_{k1} & \cdots & g_{kn} \end{bmatrix} \quad (2.2)$$

$$\mathbf{c} = \mathbf{i} * \mathbf{G} \quad (2.3)$$

Zbog svojstva generišuće matrice koje kaže da su njene vrste linearno nezavisne, možemo zaključiti da će svaki različit informacioni vektor imati različitu kodnu reč. Kodna reč  $\mathbf{c}$  dobijena je sabiranjem vrsta matrice  $\mathbf{G}$  čiji je redni broj jednak pozicijama na kojima se nalaze jedinice u informacionom vektoru  $\mathbf{i}$ . Ukupan broj ovakvih kodnih reči je zbir kombinacija koje nastaju sabiranjem pojedinačnih vrsta, zatim po dve vrste, po tri vrste itd. Matematički zapisano:

$$\binom{k}{0} + \binom{k}{1} + \binom{k}{2} + \cdots + \binom{k}{k} = 2^k \quad (2.4)$$

Ovo je takođe i broj svih vektora u kodnom potprostoru. Ako sa  $\mathbf{I}_k$  označimo jediničnu matricu dimenzija  $k \times k$ , a sa  $\mathbf{P}$  matricu dimenzija  $k \times (n-k)$ , zamenom mesta kolona i elementarnim operacijama nad vrstama generišuće matrice, dobijamo novu generišuću matricu ekvivalentnog koda, oblika:

$$G = \begin{bmatrix} 1 & 0 & \dots & 0 & p_{11} & \dots & p_{1,n-k} \\ 0 & 1 & \dots & 0 & p_{21} & \dots & p_{2,n-k} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 & p_{k1} & \dots & p_{k,n-k} \end{bmatrix} = [I_k \quad P] \quad (2.5)$$

Na ovaj način dobijamo sistematski kod, kod kod koga informacioni biti u kodnoj reči ostaju nepromenjeni, a preostali biti ( $n-k$  bita) predstavljaju proveru na parnost po bitima gde odgovarajuća kolona (kolona koja odgovara poziciji informacionog bita) matrice  $P$  ima jedinice. Problem kodera je rešen na ovaj način. Za kodovanje je dovoljno imati matricu  $G$ . Sada treba rešiti problem prijema vektora. Pitanje je na koji način proveriti da li je primljeni vektor  $r$  zapravo poslati vektor  $c$ . Da bismo ovo razrešili, uvodimo pojam **ortogonalnih potprostora**. Ako su svi vektori jednog potprostora ortogonalni svim vektorima drugog potprostora (skalarni proizvod im je jednak 0), onda su i ta dva potprostora ortogonalna. Ortogonalnost potprostora se proverava tako što se proveru ortogonalnost vektora iz njegove baze. Svaki potprostor ima svoj ortogonalni potprostor.

Neka je vektorski prostor dimenzije  $n$  i jedan potprostor dimenzije  $k$ , tada će njemu ortogonalan potprostor biti dimenzije  $n-k$ . Ako je matrica  $G$  generišuća matrica prvog potprostora, dimenzija  $k \times n$ , onda je generišuća matrica drugog potprostora,  $H$ , dimenzija  $(n-k) \times n$ . Svaki element  $a_{ij}$ , proizvoda ove dve matrice, dobija se skalarnim proizvodom  $i$ -te vrste matrice  $G$  i  $j$ -te kolone matrice  $H$ .

$$G * H^T = 0 \quad (2.6)$$

$$H * G^T = 0^T \quad (2.7)$$

Matrica  $H$  se još naziva i **kontrolna matrica**  $H$  (parity-check matrix) i ima rang  $n-k$ . Da bi se proverile greške na prijemu vektora  $r$ , treba proveriti da li on pripada ortogonalnom potprostoru čija je generišuća matrica  $H$ .

$$H = [-P^T \quad I_{n-k}] \quad (2.8)$$

Jednakost (2.7) važi ako je matrica  $G$  napisana u obliku (2.5), koji odgovara sistematskom kodu. Znak „-“ znači da za svaki element matrice  $P$  treba uzeti njemu suprotni element iz polja. Za binarni kod, svaki element, 0 i 1, sam je sebi suprotan element i ne mora se pisati znak „-“. Jedinica u matrici  $I_{n-k}$  ukazuje na poziciju kontrolnog bita, a prvih  $n-k$  elemenata svake vrste matrice  $H$  predstavlja proveru parnosti. Ukoliko se u matrici  $G$  zamene mesta matrica, važe sledeće jednakosti:

$$G_s = [P \quad I_k] \quad (2.9)$$

$$H_s = [-P^T \quad I_{n-k}] \quad (2.10)$$

**Dualni kodovi** su kodovi za koje zbog uzajamne ortogonalnosti potprostora važi sledeće:  $H$  može biti generišuća matrica linearnog koda  $(n, n-k)$  a  $G$  njegova kontrolna matrica, ako je  $G$  generišuća matrica linearnog koda  $(n, k)$  i  $H$  odgovarajuća kontrolna matrica .

### 2.3 Verovatnoća greške koja se ne može detektovati

Ako je  $c$  kodna reč koja se prenosi, a  $e$  greška prilikom prenosa, onda je primljena reč  $r$  jednaka:

$$r = c + e \quad (2.11)$$

Ako je  $H$  kontrolna matrica koda, tada važi:

$$r * H^T = (c + e) * H^T = c * H^T + e * H^T = 0 + e * H^T = e * H^T \quad (2.12)$$

Jednakost (2.12) uvodi jednu veoma važnu činjenicu, da  $r * H^T$ , proizvod koji se još naziva i **sindrom**, zavisi samo od vektora greške  $e$ , a ne od prenesene kodne reči  $c$ . Sindrom je vektor dužine  $n-k$  i od velike važnosti je za otkrivanje i ispravljanje grešaka na prijemu.

Elementi sindroma su rezultati provera na parnost, a ako su svi jednaki nula ( $n-k$  nula), onda to znači da nije bilo greške prilikom prenosa kodne reči. Ako sindrom ima bar jednu jedinicu, smatra se da je došlo do grešaka pri prenosu. Dve osnovne tehnike koje se koriste za kontrolu grešaka su Forward Error Control (FEC) i Automatic Repeat Request (ARQ).

U ARQ sistemima koristi se kod samo za detekciju grešaka. Nakon prijema kodne reči, prijemnik računa sindrom. Ako je sindrom jednak nuli, pretpostavlja se da je primljena ispravna kodna reč. Primljeni vektor se prihvata i isporučuje korisniku, a preko kanala se šalje predajniku signal da je vektor uspešno primljen. Ako je sindrom različit od nule, pretpostavlja se da je došlo do greške i prijemnik preko kanala signalizira predajniku da izvrši retransmisiju. Ovaj postupak se ponavlja sve dok vektor ne bude uspešno primljen. Korisniku se prosleđuje pogrešan vektor jedino ako prijemnik ne uspe da detektuje da je došlo do greške. Ovo se rešava upotrebom dobrog koda koji smanjuje na minimum verovatnoću da greška neće biti detektovana.

U FEC sistemima kod se koristi za detekciju i korekciju grešaka. Kada prijemnik detektuje da je došlo do greške, on pokušava da otkrije i lokaciju greške, kao i da je koriguje u isto vreme. Ako otkrije uspešno lokaciju greške, onda će uspeti i da je koriguje, dok u suprotnom, primljeni vektor biće pogrešno dekodovan i takav isporučen korisniku.

Performanse nekog zaštitnog koda primenjenog na kanalu opisuju se preko verovatnoće greške koja se ne detektuje ( $P_u$ ). U daljem tekstu biće analizirana verovatnoća greške koja se ne može detektovati za poseban oblik kanala, **binarni smetrični kanal** (BSC) koji je definisan sledećom matricom:

$$P_{BSC} = \begin{bmatrix} v & p \\ p & v \end{bmatrix}, \quad p + v = 1 \quad (2.13)$$

Kod BSC kanala, pod simetrijom se podrazumeva da je verovatnoća prelaska iz 0 u 1 jednaka verovatnoći prelaska iz 1 u 0.

$$p(0 \rightarrow 1) = p(1 \rightarrow 0) \quad (2.14)$$

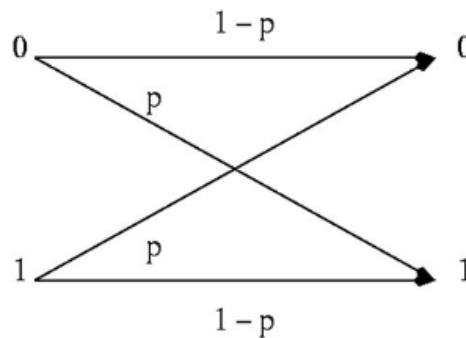
Ako parametar  $v$  izrazimo kao  $1 - p$ , onda se ovaj kanal može opisati samo preko jednog parametra,  $p$  (slika 2.2). Ovaj parametar ima vrednost jednaku verovatnoći greške ( $P_e$ ). U ovom slučaju, verovatnoća greške ne zavisi od verovatnoća pojavljivanja simbola na ulazu u

kanal. Kod realnog BSC kanala greške se javljaju statistički nezavisno, to je kanal bez interferencije simbola.

Greške se ne detektuju samo onda kada se vektor greške ( $e$ ) poklapa sa nenultom kodnom reči. Raspodele težina linearnog blok koda potrebne su za otkrivanje verovatnoće greške koja se ne može detektovati ( $P_u$ ). Raspodela težina predstavlja skup brojeva  $\{A_0, A_1, A_2 \dots A_n\}$ , takav da  $A_i$  označava broj kodnih koje imaju Hemingovu težinu  $i$ . Verovatnoća nedetektovane greške može se predstaviti sledećim izrazom:

$$P_u = \sum_{i=1}^n A_i p^i (1-p)^{n-i} \quad (2.15)$$

$$P_u = (1-p)^n \sum_{i=1}^n A_i \left(\frac{p}{1-p}\right)^i \quad (2.16)$$



Slika 2.2 Graf BSC kanala

$A(z)$  predstavlja spektar kodnih rastojajna i može se napisati u sledećem obliku:

$$A(z) = A_0 + A_1 z + A_2 z^2 + \dots + A_n z^n \quad (2.17)$$

Neka  $B(z)$  predstavlja spektar dualnog koda koji ima  $n-k$  informacionih bita. Ako  $k$  nije malo, pretraga  $A(z)$  može veoma dugo da traje, pa je jednostavnije naći  $B(z)$ . Vezu između spektra

dualnih kodova predstavljaju **Makvilijamsovi identiteti**. Sledeće jednačine važe za  $A(z)$  i  $B(z)$ , pri čemu je u drugoj primenjeno da je  $A_0 = 1$  i  $z = \frac{p}{1-p}$ :

$$A(z) = 2^{-(n-k)}(1+z)^n B\left(\frac{1-z}{1+z}\right) \quad (2.18)$$

$$P_u(p) = (1-p)^n \left[ A\left(\frac{p}{1-p}\right) - 1 \right] \quad (2.19)$$

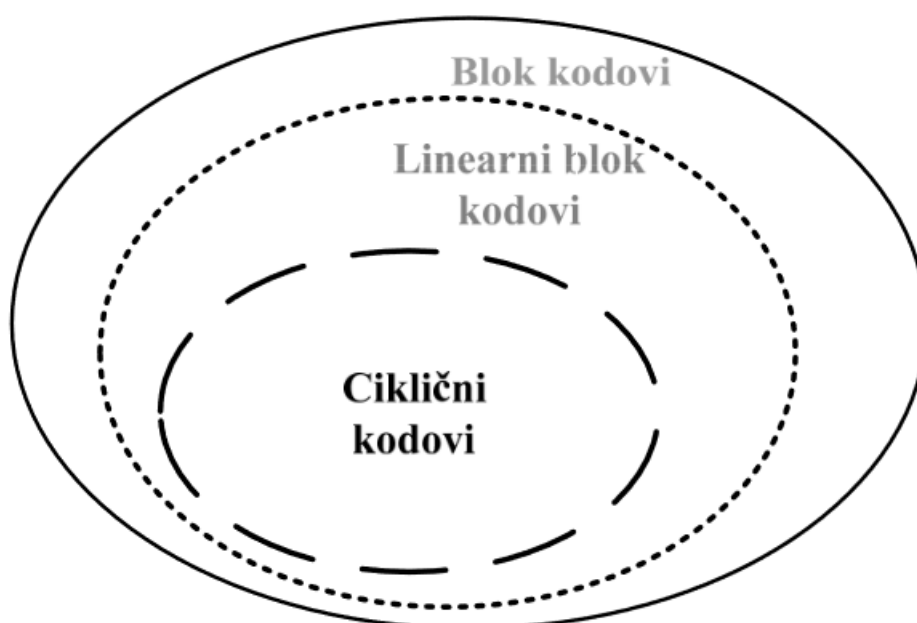
$$B(1-2p) = \sum_{i=0}^n B_i(1-2p)^i \quad (2.20)$$

$$P_u(p) = 2^{-(n-k)} B(1-2p) - (1-p)^n \quad (2.21)$$

Određivanje spektra na ovakav način može biti veoma teško ako su  $n$ ,  $k$  i  $n-k$  velike vrednosti. Međutim, ako je nemoguće izračunati tačnu vrednost verovatnoće greške, moguće je naći njenu gornju granicu usrednjenu po svim  $(n, k)$  linearnim kodovima.

### 3 CIKLIČNI KODOVI

Ciklični kodovi su potklasa linearnih blok kodova, i od velike važnosti su u savremenim telekomunikacionim sistemima.



Slika 3.1 Šematski prikaz potklase Ciklični kodovi

#### 3.1 Uvod

Opis cikličnih kodova je još jednostavniji od opisa linearnih blok kodova. Zbog svoje jednostavne hardverske implementacije, ciklični kodovi su klasa linearnih kodova koja se koristi i za otkrivanje i za ispravljanje grešaka (pogotovo jednostrukih grešaka). Kodne reči cikličnih kodova predstavljaju se kao polinomi sa koeficijentima iz konačnog polja. Koeficijente polinoma po proizvoljnoj promenljivoj  $x$ , predstavljaju biti (simboli) kodne reči (vektora). Elementi su u vektoru uređeni po svojim pozicijama, a u polinomu su uređeni stepenima promenljive. Zato važi sledeće poređenje vektora  $v$  i polinoma  $v(x)$ :

$$v(a_0, a_1, \dots, a_{n-1}) \leftrightarrow v(x) = a_{n-1}x^{n-1} + \dots + a_1x + a_0 \quad (3.1)$$

Neka je dat vektorski prostor dimenzije  $n$ . Cikličan kod je njegov potprostor ako za bilo koji vektor  $v(a_0, a_1, \dots, a_{n-1})$  važi da vektor  $v1(a_{n-1}, a_0, a_1, \dots, a_{n-2})$ , koji je dobijen cikličnom permutacijom koeficijenata, takođe pripada ovom kodnom potprostoru.

**Primer 3.1:** Predstaviti vektor  $v$  polinomom  $v(x)$

$$v = 100011 \rightarrow v(x) = x^5 + x + 1$$

$$v = 001010101 \rightarrow v(x) = x^6 + x^4 + x^2 + 1$$

Polinomi se međusobno mogu sabirati, množiti i nad njima se može vršiti množenje konstantom (elementom polja). Što se tiče međusobnog sabiranja polinoma, sabiraju se koeficijenti uz odgovarajuće stepene promenljive. Kod međusobnog množenja koeficijenti se dobijaju kao zbir proizvoda koeficijenata uz odgovarajuće stepene promenljive, ali ovde može doći do problema pošto najviši stepen proizvoda dva polinoma može biti veći od  $n-1$ . Da bi se izbegao prelazak u vektorski prostor većih dimenzija, uvodi se množenje polinoma po modulu polinoma stepena  $n$ . Na ovaj način kao ostaci se dobijaju polinomi čiji stepen ne prelazi vrednost  $n-1$ . Za polje  $GF(2)$ , takvih polinoma ima  $2^n$ , isto koliko i vektora. Hardverska realizacija koda i dekodera postala je dosta jednostavnija uvođenjem ovakvog množenja polinoma.

**Generišući polinom** koda,  $g(x)$ , je polinom koji deli sve polinome koji čine jedan cikličan potprostor, takođe deli i polinom  $x^n-1$ . Ciklični kod je ideal u algebri polinoma po modulu  $x^n-1$  nad poljem koeficijenata. Bilo koji polinom generiše svoj kod ako je delitelj  $x^n-1$ . Ako je polinom  $g(x)$  stepena  $n-k$ , onda su linearno nezavisne klase ostataka, polinomi:

$$0, g(x), xg(x), x^2g(x), \dots, x^{k-1}g(x) \quad (3.2)$$

Vektori ovih polinoma i njihove linearne kombinacije čine cikličan kod. Polinom  $x^k g(x)$  biće stepena  $n$ , ako je  $g(x)$  stepena  $n-k$ , i biće predstavljen svojim ostatkom pri deljenju sa  $x^n - 1$ . Kako važi da je  $x^n = 1$ , zaključujemo da cikličan pomeraj koeficijenata polinoma odgovara množenju polinoma sa  $x$ , jer važi:

$$x(a_0 + a_1x + \dots + a_{n-1}x^{n-1}) = a_{n-1} + a_0x + \dots + a_{n-2}x^{n-1} \quad (3.3)$$

Polinom klase ostataka pripada kodu ako važi:

$$x^n - 1 = g(x)h(x) \quad (3.4)$$

Polinom  $h(x)$ , naziva se još i **kontrolni polinom**, on takođe generiše svoj ideal, dualni kod  $(n, n-k)$ . Kod  $(n, k)$  dobija se ako je generišući polinom stepena  $n-k$ , a kontrolni polinom stepena  $k$ .

Da bismo videli koji su svi mogući ciklični kodovi za prethodno navedene uslove, moramo uvesti pojam nesvodljivog polinoma. **Nesvodljiv polinom** je polinom koji se ne može dalje faktorizovati u datom polju. Polinom  $x^n - 1$  treba razložiti na nesvodljive polinome, tada svaki taj polinom ima svoj ideal i sve kombinacije proizvoda polinoma takođe daju svoje ideale. To su svi mogući ciklični kodovi pod datim uslovima. Ciklični kodovi su veoma malobrojni u odnosu na linearne i blok kodove istih dimenzija. Za  $(n, k) = (7, 3)$  važi da postoji  $1.34 \cdot 10^{12}$  blok kodova, 11811 linearnih blok kodova, a samo 2 ciklična linearna blok koda.

Dok je za opis blok koda  $(n, k)$  potrebno znati  $2^k$  njegovih kodnih reči dužine  $n$ , a za opis linearnog blok koda sa istim parametrima potrebno je znati njegov spektar (bazu),  $k$  kodnih reči dužine, važi to da je za cikličan kod dovoljno znati jednu kodnu reč dužine  $n$ , a sve ostale kodne reči dobijaju se kao cikličan pomeraj poznate kodne reči i kao linearna kombinacija ovih permutacija. U ovome se ogleda jednostavnost cikličnih kodova u poređenju sa linearnim blok kodovima.

### 3.2 Primeri

**Primer 3.2:** Faktorisati polinom  $x^7 + 1$ , odabrati odgovarajući generišući polinom, prikazati generišuću matricu, kontrolni polinom i kontrolnu matricu.

$$x^7 + 1 = (1 + x)(1 + x + x^3)(1 + x^2 + x^3) \quad (n = 7)$$

Za generišući polinom uzimamo:

$$g(x) = 1 + x^2 + x^3 \quad (n - k = 3)$$

Sada smo dobili kod (7,4)  $n = 7$ ,  $k = 4$ . Prvu kodnu reč možemo dobiti ošitavanjem koeficijenata generišućeg polinoma:

$$g(x) = (1011000)$$

Za generišuću matricu biramo još tri linearno nezavisne kodne reči, koje ćemo dobiti cikličnim pomeranjem prve reči:

$$xg(x) = (0101100)$$

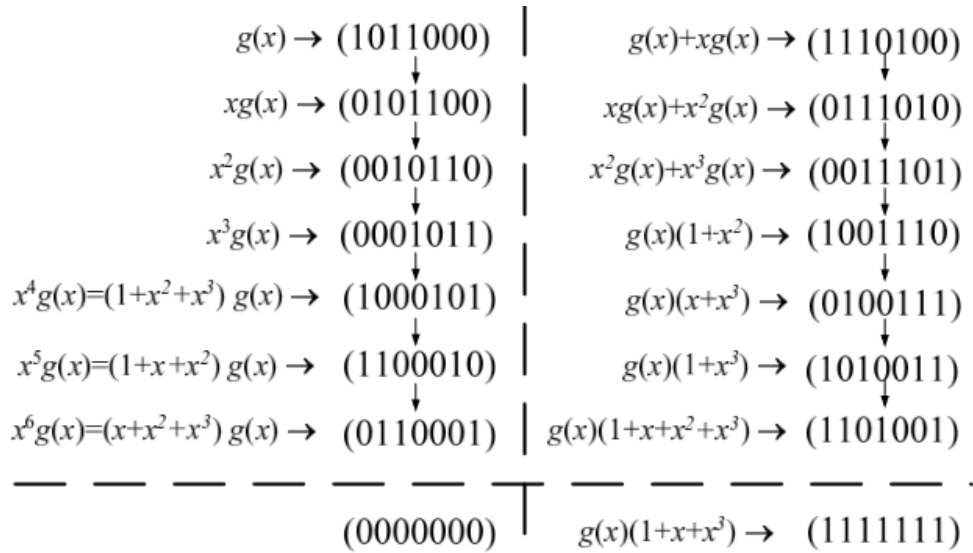
$$x^2g(x) = (0010110)$$

$$x^3g(x) = (0001011)$$

Generišuća matrica:

$$G = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

Na slici 3.2 možemo videti sve kodne reči ovog cikličnog koda i način njihovog formiranja preko polinoma.



Slika 3.2 Ilustracija cikličnosti koda (7,4)

Kontrolni polinom, dobijen množenjem preostalih faktora:

$$h(x) = (1+x)(1+x+x^3) = 1+x^2+x^3+x^4$$

Sada izražavamo kodne reči:

$$\begin{aligned} h(x) &= (1011100) \\ xh(x) &= (0101110) \\ x^2h(x) &= (0010111) \end{aligned}$$

Kontrolna matrica:

$$H = \begin{bmatrix} 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 \end{bmatrix}$$

Lako je proveriti da važi:

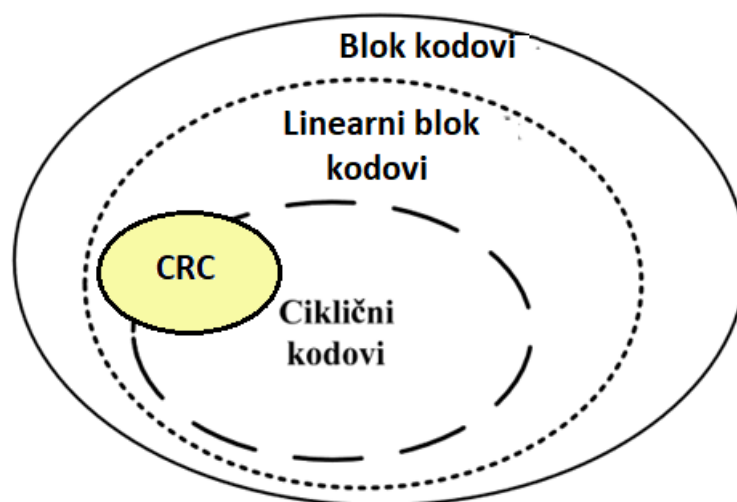
$$GH^T = 0$$

Kod linearnih kodova smo spomenuli da se informaciona poruka koduje množenjem informacionog vektora i generišuće matrice, međutim kod cikličnih kodova možemo matricu zameniti generišućim polinomom. Kao rezultat množenja ova dva polinoma, dobijamo polinom koji će predstavljati kodnu reč. Kodovanje na ovakav način (množenjem generišućim polinomom) dosta je jednostavnije realizovati, i hardverski i softverski. Od hardverskih komponenti koriste se linearna prekidačka kola kojim se može vršiti množenje i deljenje polinoma. Nekada je pogodnije vršiti množenje, a nekada deljenje, a u nastavku je objašnjenje od čega zavisi izbor operacije.

Ciklični kod je u potpunosti opisan generišućim polinomom,  $g(x)$ . Dakle, da bi se dobio ciklični kod treba od informacionog polinoma  $i(x)$  (stepena  $k-1$ ), dobiti kodni polinom  $c(x)$  (stepena  $n-1$ ), koji je deljiv generišućim polinomom  $g(x)$ . Na prijemu treba primljeni polinom  $r(x)$  podeliti polinomom  $g(x)$ , ako je ostatak ravan nuli, smatra se da nije bilo grešaka. Takođe, na prijemu se polinom  $r(x)$  može pomnožiti sa  $h(x)$  i proveriti da li je rezultat ravan nuli (po modulu  $x^n-1$ ). Pri deljenju polinomom stepena  $n$ , šema sadrži pomerački registar koji ima  $n$  ćelija. U slučaju množenja polinomom stepena  $n$ , šema sadrži pomerački registar  $n$  ćelija  $k$ . Pošto je  $g(x)$  stepena  $n-k$ , registar za deljenje ima  $n-k$  ćelija. Ako se vrši množenje polinomom  $h(x)$  stepena  $k$  i registar za množenje će imati  $k$  ćelija. Kod dugačkih kodova, sa visokom vrednošću stepena  $k$ , dosta je isplativije raditi deljenje polinoma, jer onda registar ima  $n-k$  ćelija. U daljem objašnjenju, videćemo da je to upravo razlog zbog koga se koristi deljenje kod CRC postupka.

## 4 CIKLIČNA PROVERA REDUNDANSE (CRC)

CRC kodovi predstavljaju potklasu linearnih kodova. Oni nemaju mogućnost ispravljanja greške, već samo mogu da je detektuju, služe kao finalna provera da li je poslata informacija ispravno primljena. Mogu se koristiti i u kombinaciji sa drugim tehnikama zaštitnog kodovanja koje mogu da ispravljaju greške.



Slika 4.1 Šematski prikaz CRC potklase

### 4.1 Uvod

CRC kod, u oznaci  $(n, k)$  u potpunosti je opisan generišućim polinomom  $g(x)$ . Kod ovih kodova, na početak ili na kraj binarne informacione poruke (dužine  $k$  bita) dodaje se  $n - k$  bita za proveru parnosti. Na taj način se dobijaju kodne reči dužine  $n$ , deljive sa generišućim polinomom. Ovo predstavlja veliku prednost, jer se mogu koristiti u sistemima koji koriste različite dužine kodne reči.

Nije svaki CRC kod cikličan, u opštem slučaju. Samo specijalni slučajevi, čiji generišući polinom deli polinom  $x^n - 1$  ( $n$  je dužine kodne reči), spadaju u ciklične kodove.

## 4.2 CRC postupak

Neka je dat polinom informacionih bita,  $i(x)$ , stepena  $k-1$  i generišući polinom,  $g(x)$ , stepena  $n-k$ . Prvo ćemo pomnožiti polinom  $i(x)$  sa  $x^{n-k}$ , tako da dobijemo novi informacioni polinom,  $i'(x)$ , stepena  $n-1$ . Zatim  $i'(x)$  delimo generišućim polinomom i dobijamo polinom ostatka  $r(x)$  kome maksimalan stepen može biti  $n-k-1$ . Konačno, kodnu reč,  $c(x)$ , dobijamo sabiranjem polinoma  $i'(x)$  i ostatka  $r(x)$ . Informacioni biti ostaju neizmenjeni, a dodati biti predstavljaju bite za proveru parnosti.

$$i(x) = i_{k-1}x^{k-1} + \dots + i_1x + i_0 \quad (4.1)$$

$$g(x) = g_{n-k}x^{n-k} + \dots + g_1x + g_0 \quad (4.2)$$

$$i'(x) = x^{n-k}i(x) = i_{k-1}x^{n-1} + \dots + i_1x^{n-k-1} + i_0x^{n-k} \quad (4.3)$$

$$r(x) = r_{n-k-1}x^{n-k-1} + \dots + r_1x + r_0 \quad (4.4)$$

$$c(x) = x^{n-k}i(x) + r(x) \quad (4.5)$$

$$c(x) = x^{n-k}i(x) + \text{rem}\left\{\frac{x^{n-k}i(x)}{g(x)}\right\} \quad (4.6)$$

$\text{Rem}\{\cdot\}$  predstavlja ostatak pri deljenju polinoma. Na prethodno opisani način formiran je transmisioni ram, na koji se može primeniti neki drugi zaštitni kod koji ima mogućnost ispravljanja grešaka.

Neka je  $e(x)$  polinom greške koji se superponira na bite transmisionog rama pri prenosu kroz kanal. Ovakav primljeni polinom se deli generišućim polinomom, a ostatak pri deljenju je jednak nuli ako nije bilo grešaka pri prenosu. Ako je ostatak različit od nule, možemo zaključiti da je bilo grešaka, ali ne i na kojoj poziciji. Ostatak pri deljenju je oblika:

$$r'(x) = \text{rem} \left\{ \frac{c(x)+e(x)}{g(x)} \right\} \quad (4.7)$$

Dužina rama može biti promenljiva, pa se može pribeći skraćivanju cikličnog koda. Pošto se smatra da najstariji informacioni biti imaju vrednost nula i ne utiču na ostatak pri deljenju, oni se ne moraju slati već se odsecaju.

### 4.3 Performanse CRC postupka

Performanse CRC koda zavise od procenta vektora grešaka koje se mogu detektovati. Dekoder može da pogreši u slučaju da mu na ulaz stigne sekvenca bita koja odgovara kodnoj reči, a zapravo je nastala superponiranjem sekvence grešaka sa prvobitnom kodnom reči. U ovom slučaju, dekodeer neće otkriti da je došlo do greške (a(x) je neki nenulti polinom):

$$e(x) = g(x)a(x) \quad (4.8)$$

Za statistički nezavisne greške važi da CRC kod može detektovati sve jednostruke greške ako je polinom  $e(x)$  izabran takav da su mu bar dva koeficijenta različita od nule. Polinom  $p(x) = x + 1$  služi za otkrivanje svih neparnih grešaka i predstavlja opštu proveru parnosti. U praksi se najčešće za generišući polinom bira proizvod prethodno navedenog polinoma  $p(x)$  i drugog polinoma, stepena  $n-k-1$ . Verovatnoća nedetektovane greške zavisi od statistike grešaka u kanalu. Sa porastom faktora  $n-k$  raste i sposobnost koda da otkrije greške, jer na ulaz dekodeera može doći ukupno  $2^n$  binarnih kombinacija, od kojih su  $2^k$  kodne reči.

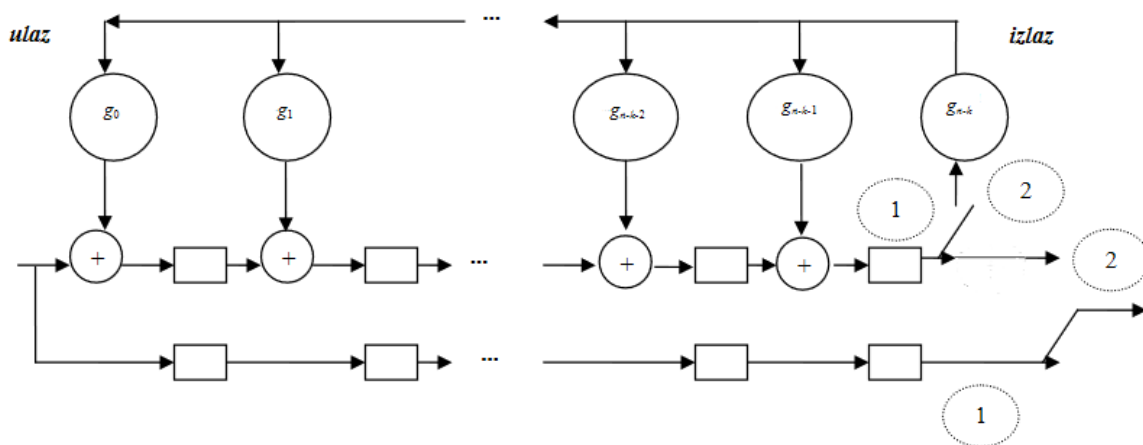
Za paketske greške, koje se javljaju u kanalima sa memorijom, pri prenosu fajlova, važi da vektor greške sadrži tačno određen broj uzastopnih jedinica (m), pa onda idu nule. Sada se vektor greške može napisati u sledećem obliku (polinom  $e'(x)$  je stepena  $m - 1$ , a  $i$  je pozicija najmanjeg nenultog koeficijenta u polinomu  $e(x)$ ):

$$e(x) = x^i e'(x) \quad (4.9)$$

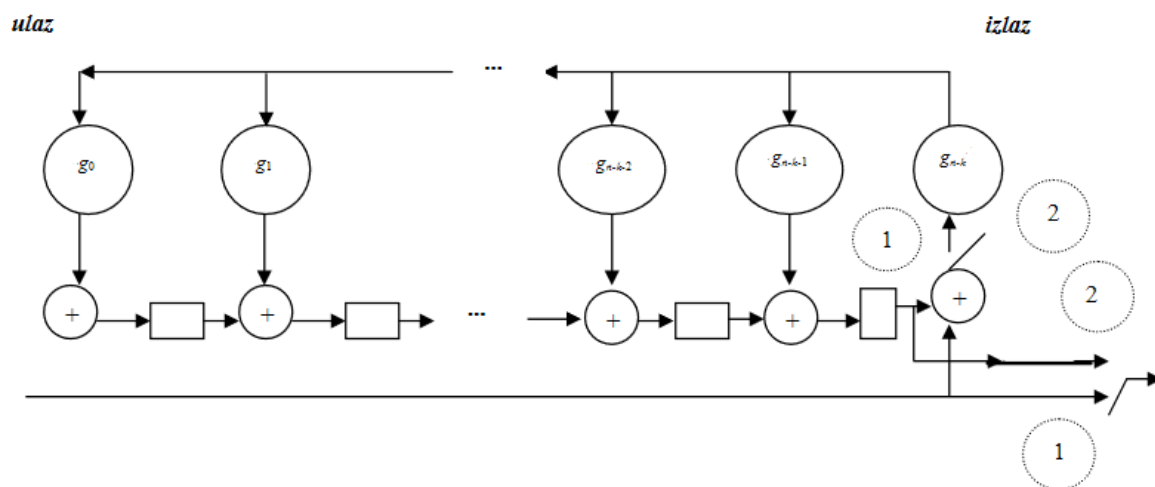
Iz ovoga sledi zaključak da CRC kod može detektovati sve paketske greške ne veće dužine od vrednosti najvišeg stepena generišućeg polinoma. Ovo se može dokazati na sledeći način: znamo da se greške ne detektuju ako je polinom greške deljiv sa  $g(x)$  i da  $x^i$  sigurno nije deljivo sa  $g(x)$ , a ako je  $m = n-k$ , onda polinom  $e'(x)$  ima maksimalan stepen  $n-k-1$ , pa ni on nije deljiv sa  $g(x)$ .

Na slici 4.2 nalazi se šema dobijanja CRC koda. Na početku rada prekidači su u položaju (1). K informacionih bita u isto vreme ulazi u registar i šalje se na liniju. Donja grana ima  $n-k$  ćelija, što odgovara množenju sa  $x^{n-k}$ . Sada kada su informacioni biti otišli na liniju, prekidači se prebacuju u položaj (2). Ostatak deljenja iz gornjeg registra se očitava na liniji. Na ulaz se u isto vreme ne dovodi ništa, može se staviti prekidač ili se mogu dovoditi "nule". Dodatno kašnjenje, odnosno donji registar, može se izbeći šemom prikazanom na slici 4.3. Ova šema daje isti rezultat kao šema sa slike 4.2, ali bez kašnjenja.

Na početku rada prekidači su u položaju (1). Informacioni biti idu odmah direktno na liniju, a u registru se formira ostatak. Sadržaj registra u toku rada neće biti identičan sadržajima registra na prethodnoj šemi, ali će na kraju slanja informacionih bita na liniju sadržaji oba registra biti identični. Sada se prekidači menjaju u položaj (2)

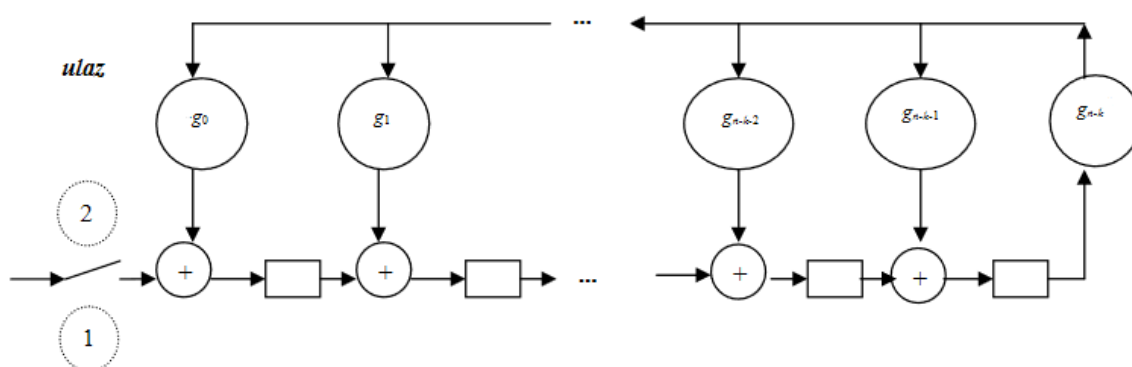


Slika 4.2 Šematski prikaz dobijanja CRC koda



Slika 4.3 Šematski prikaz koda CRC koda bez kašnjenja

Jedan od načina je da se na prijemu proveri da li je primljeni polinom kodni ili nije – je provera da li je deljiv sa  $g(x)$ . Ako je ostatak pri deljenju ravan nuli, smatra se da nije bilo grešaka. Šema dekodera prikazana na slici 4.4 može se koristiti za ovu proveru. U kolo za deljenje uvodi se primljeni vektor. Posle  $n$  koraka pregleda se registar, u kome se sada nalazi ostatak od deljenja sa  $g(x)$ . Ovaj ostatak predstavlja sindrom koji se sada može shvatiti i kao polinom  $s(x)$ . Smatra se da nije došlo do grešaka pri prenosu, ako je ostatak ravan nuli. Sadržaj primljenog vektora se uništava ovakvim deljenjem, pa ga treba čuvati u još jednom baferu, da bi se mogli izdvojiti informacioni simboli. Ako sindrom nije jednak nuli, a smatra se da se dogodila samo jedna greška u kodnoj reči, tada se ovom šemom može odrediti i njena pozicija.



Slika 4.4 Šematski prikaz dekodera CRC koda

## **5 ANALIZA SIMULACIJE U PROGRAMU**

Ovo poglavlje sadrži opis postupka simulacije pojedinih CRC kodova koji su dati u tabeli 5.1, kao i opis protokola koji će biti korišćeni u simulaciji. Kod programa u kome je vršena simulacija napisan je na programskom jeziku C++ i može se naći u Dodatku.

### **5.1 Opis protokola korišćenih u simulaciji**

#### **5.1.1 Bluetooth**

#### **5.1.2. USB**

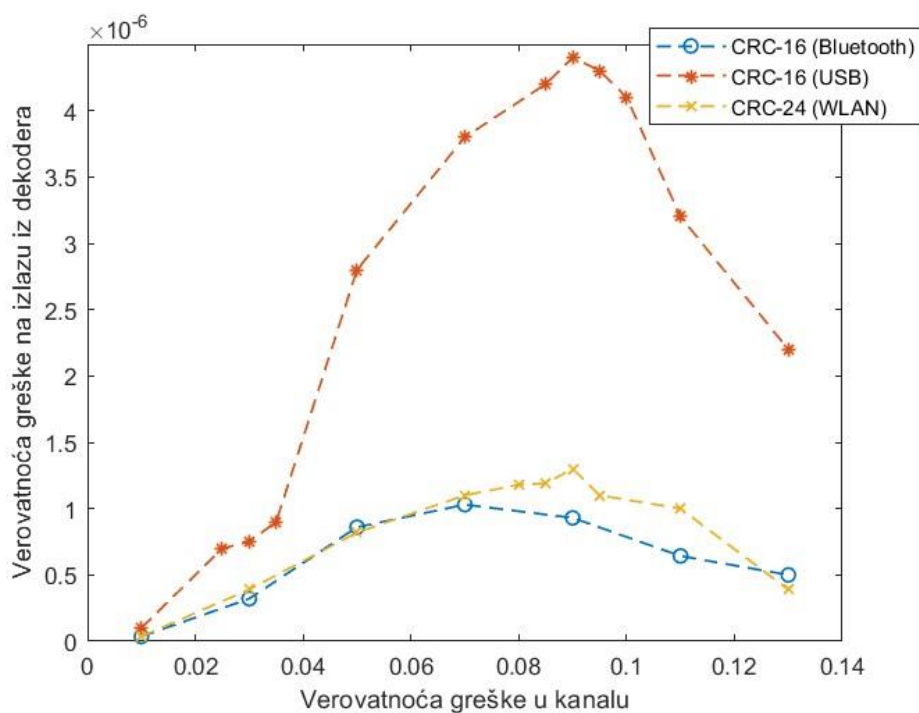
#### **5.1.3. WLAN**

## 5.2 Opis simulacije

Tabela 5.1 Kodovi za simulaciju i njihovi generišući polinomi

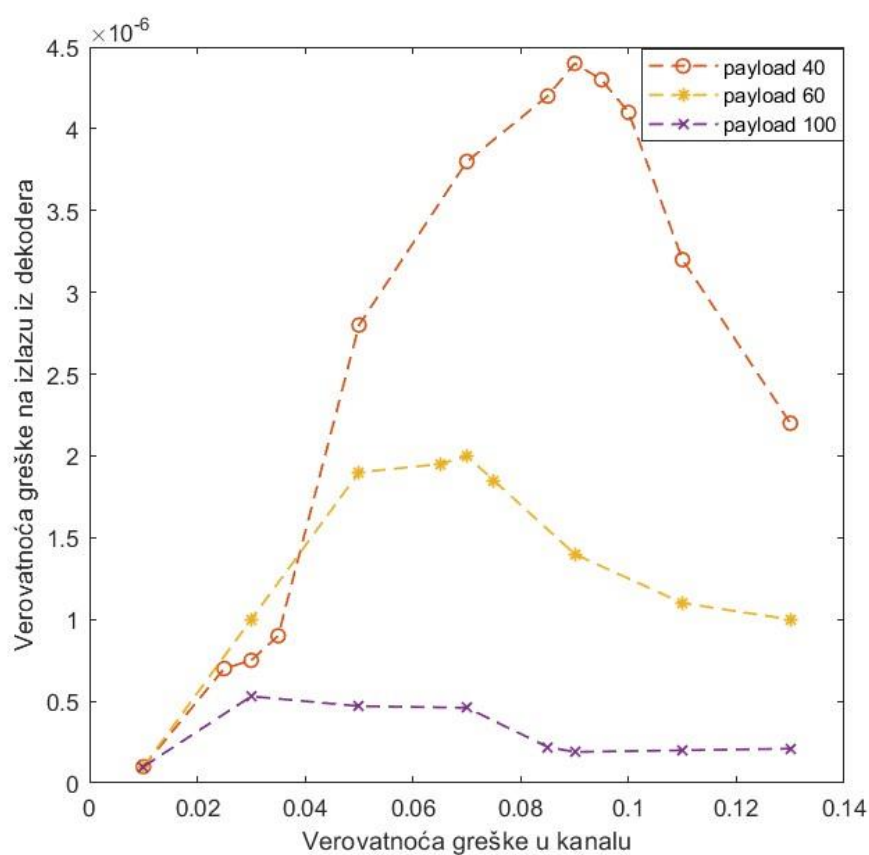
| Naziv            | Generišući polinom          |
|------------------|-----------------------------|
| CRC-16 Bluetooth | $x^{16}+x^{12}+x^5+1$       |
| CRC-16 USB       | $x^{16}+x^{15}+x^2+1$       |
| CRC-24 WLAN      | $x^{24}+x^{23}+x^6+x^5+x+1$ |

Na slici 5.1 prikazani su kodovi CRC-24 WLAN, CRC-16 USB i CRC-16 Bluetooth. Vidi se zavisnost greške na izlazu iz dekodera i greške u kanalu, verovatnoća greške u kanalu je uzeta u opsegu [0.01,0.13]. Sa slike se može videti da CRC-24 WLAN ima slično ponašanje kao CRC-16 Bluetooth, ali znatno bolje od CRC-16 USB koda.

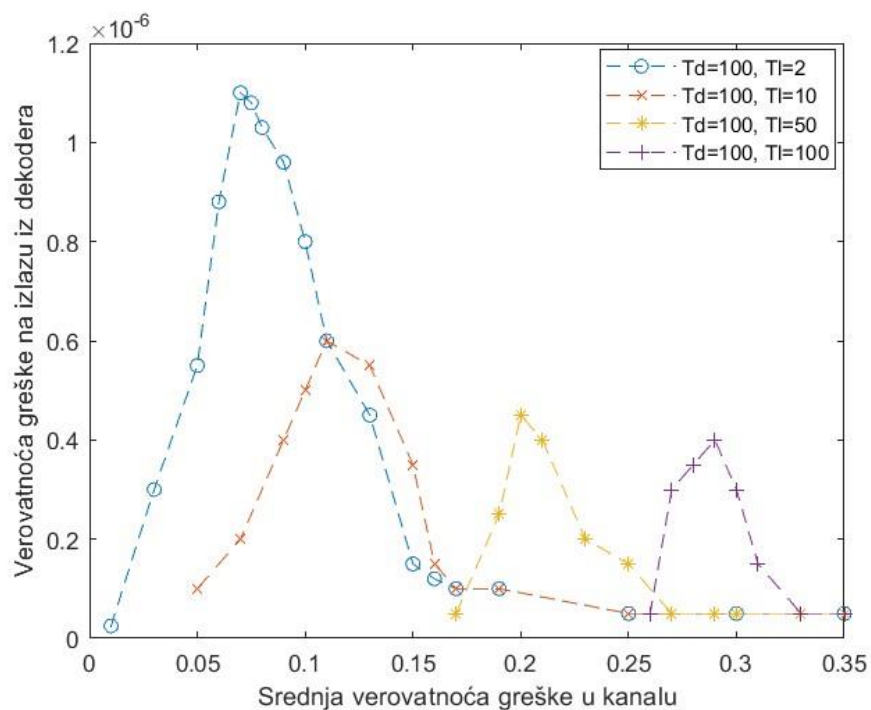


Slika 5.1 Zavisnost greške na izlazu iz dekodera i greške u kanalu

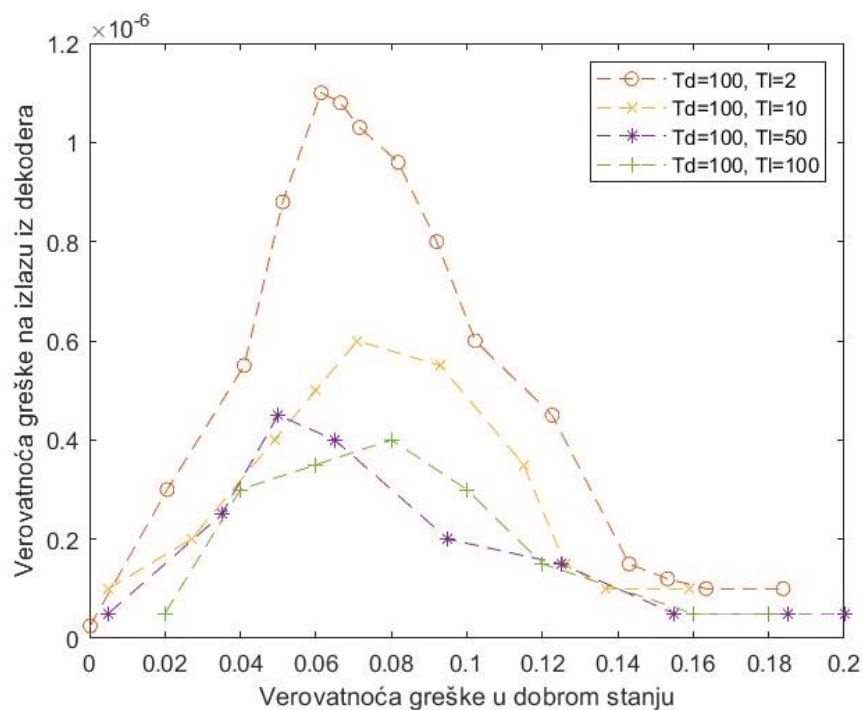
Takođe, na slici 5.2 možemo videti ponašanje koda CRC-16 USB za različite vrednosti parametra payloadSize (koji je tačan prevod parametra payload?). Najbolje rezultate ima za payloadSize = 100.



Slika 5.2 Ponašanje koda CRC-16 USB za različite vrednosti parametra payloadSize



Slika 5.3 Zavisnost greške na izlazu iz dekodera srednje verovatnoće greške, u Gilbert-Eliotovom kanalu



Slika 5.4 Zavisnost greške na izlazu iz dekodera verovatnoće greške u dobrom stanju, u Gilbert-Eliotovom kanalu

## **6 ZAKLJUČAK**

## 7 LITERATURA

[1]

[2]

## 8 DODATAK

Ovde će biti izložen programski kod korišćen za simuliranje CRC koda.

Config.h

```
1  #ifndef CONFIG_H
2      #define CONFIG_H
3
4
5      // struktura za konfigurisanje simulacije
6
7  struct Config
8  {
9      //parametri izvora
10     const double transProb01 = 0.2;
11     const double transProb10 = 0.5;
12     const int sourceStreamSize = 4000000;
13     const int payloadSize = 40;
14     int numFrames = sourceStreamSize/payloadSize;
15
16     // parametri crc bloka
17     const int crcGenPoly[9] = { 1,0,0,0,0,0,1,1,1 };
18     const int overheadSize = sizeof(crcGenPoly)/sizeof(int)-1;
19     const int frameLength = numFrames*(payloadSize+overheadSize);
20     int totallength;
21
22     // parametri kanala
23     const double bscErrorProbArray[1] = { 0.13 };
24     const int numOfChannels = sizeof(bscErrorProbArray)/sizeof(double);
25 };
26
27 enum userSide{transmit, receive};
28
29 #endif
```

Polynomial.h

```
1  #ifndef POLYNOMIAL_H
2  #define POLYNOMIAL_H
3
4  #include<iostream>
5  using namespace std;
6
7  class Polynomial
8  {
9      int polyDegree;    // maksimalni stepen polinoma
10     int* polyCoefficients; // vektor koeficijenata
11
12     public:
13
14     // konstruktori
15     Polynomial();    //inicijlizacija praznim polinomom
16     Polynomial(const int degree, const int* coeffs);    // inicijalizacija vektorom
17                                                         // koeficijenata
18     Polynomial(const Polynomial& poly);    // inicijalizacija drugim polinomom
19     Polynomial(Polynomial&& poly);    // inicijalizacija drugim polinomom
20
21     // destruktor
22     ~Polynomial() {delete[] polyCoefficients;};
23
24     Polynomial operator=(const Polynomial& poly);
25
26     // pristup privatnim članovima preko javnih metoda
27     int getPolyDegree() const;
28     int* getPolyCoefficients() const;
29
30     void setPolynomial(int, int*);
31
32     // ispis polinoma
33     void printPolynomial() const;
34
35     // ostatak deljenja dva polinoma se smešta u polinom deljenika, dok se kao argument
36     // uzima polinom delioca
37     void calculateReminder(Polynomial& poly);
38
39 };
40
41 #endif
```

Polynomial.cpp

```
1  #include "Polynomial.h"
2
3
4  Polynomial::Polynomial()
5  {
6      polyDegree = -1;           // nepostojeći stepen polinoma
7      polyCoefficients = nullptr; // ne postoje ni koeficijenti
8  }
9
10 Polynomial::Polynomial(int degree, const int* coeffs)
11 {
12     polyDegree = degree;
13
14     polyCoefficients = new int[polyDegree+1]; //alocira se memorija za koeficijente
15
16     for(int i=0;i<=polyDegree;i++) polyCoefficients[i] = coeffs[i];
17 }
18
19 Polynomial::Polynomial(const Polynomial& poly) //konstruktor kopije
20 {
21     polyDegree = poly.polyDegree;
22     polyCoefficients = new int[polyDegree+1];
23
24     for(int i=0;i<=polyDegree;i++) polyCoefficients[i] = poly.polyCoefficients[i];
25 }
26
27 Polynomial::Polynomial(Polynomial&& poly) //konstruktor kopije
28 {
29     polyDegree = poly.polyDegree;
30     polyCoefficients = new int[polyDegree+1];
31
32     for(int i=0;i<=polyDegree;i++) polyCoefficients[i] = poly.polyCoefficients[i];
33     poly.setPolynomial(-1,nullptr);
34 }
35
36 void Polynomial::printPolynomial() const //ispis polinoma
37 {
38     bool flag = true;
39
40     cout << "Polynomial: ";
41     for(int i=0;i<=polyDegree;i++)
42     {
43         if(polyCoefficients[i]==1)
44         {
45             if(flag){ cout << "x^" << polyDegree - i; flag = false;}
46             else cout << "+x^" << polyDegree - i;
47         }
48     }
49 }
```

```
50     if(flag) cout << 0;
51     cout << endl;
52     cout << "-----\n";
53 }
54
55 int Polynomial::getPolyDegree() const
56 {
57     return this->polyDegree; // pristup preko podrazumevanog pokazivača this (korišćenje
58                             // pokazivača this u ovom primeru nije bilo neophodno)
59 }
60
61
62 int* Polynomial::getPolyCoefficients() const
63 {
64     int* polyPtr = new int[polyDegree+1];
65     for (int i=0;i<=polyDegree;i++) polyPtr[i]=polyCoefficients[i];
66
67     return polyPtr;
68 }
69
70 void Polynomial::setPolynomial(int degree, int* polyPtr)
71 {
72
73     if(polyDegree>-1)
74     {
75
76         delete[] polyCoefficients; // brišu se prethodno korišćeni koeficijenti
77     }
78
79     polyDegree = degree; // direktan pristup privatnom članu
80
81     if(polyDegree>-1)
82     {
83         polyCoefficients = new int[polyDegree+1]; // alocira se nova memorija za
84                                                  // koeficijente
85
86         for (int i=0;i<=polyDegree;i++) polyCoefficients[i] = polyPtr[i];
87     }
88     else
89         polyCoefficients = nullptr;
90 }
91
92 Polynomial Polynomial::operator=(const Polynomial& poly)
93 {
94     polyDegree = poly.polyDegree;
95     polyCoefficients = new int[polyDegree+1];
96
97     for(int i=0;i<=polyDegree;i++) polyCoefficients[i] = poly.polyCoefficients[i];
98
99     return *this;
100 }
```

```

101
102 void Polynomial::calculateReminder(Polynomial& poly)
103 {
104     int* rem;
105     rem = new int[poly.polyDegree+1];
106
107     int* temp; //pomoći pokazivač koji pokazuje na tekući rezultat deljenja
108     temp = new int[poly.polyDegree+1];
109
110     for(int i=0;i<poly.polyDegree+1;i++)
111         rem[i]=polyCoefficients[i]; //ostatak pri deljenju se inicijalizuje na deljenik
112
113     for(int j=poly.polyDegree+1;j<=polyDegree+1;j++)
114     {
115         for(int m=0;m<poly.polyDegree+1;m++) temp[m]=rem[m];
116
117         if(rem[0]==0) // ako je najveći stepen tekućeg ostatka '0'
118             // preskače se jedan stepen deljenja
119         {
120             for(int m=0;m<poly.polyDegree;m++) rem[m]=temp[m+1];
121
122         }
123
124         else // u suprotnom se vrši XOR operacija tekućeg ostatka i deljenika
125         {
126             for(int i=0;i<poly.polyDegree;i++)
127                 rem[i]=(temp[i+1]+poly.polyCoefficients[i+1])%2;
128
129         }
130
131         if(j!=polyDegree+1)
132             rem[poly.polyDegree]=polyCoefficients[j]; //dodaje se sledeći
133             // bit deljenika
134         else
135             rem[poly.polyDegree]=0;
136     }
137
138     delete[] polyCoefficients; // sada je moguće izbrisati stare koeficijente
139     // da bi se upisali novi
140
141     polyCoefficients = new int[poly.polyDegree]; // dodela memorije za nove koeficijente
142
143     // kopiranje ostatka (rem) u nove koeficijente
144     for(int m=0;m<poly.polyDegree;m++) polyCoefficients[m]=rem[m];
145
146     delete[] rem;
147     delete[] temp;
148
149     polyDegree = poly.polyDegree-1; // novi maksimalni stepen polinoma ostatka
150
151 }

```

CrcBlock.h

```
1  #ifndef CRC_BLOCK_H
2  #define CRC_BLOCK_H
3
4
5  #include "Polynomial.h"
6
7  class CrcBlock
8  {
9      Polynomial poly;
10
11  public:
12
13      // konstruktori
14      // inicijalizacija praznim blokom
15      CrcBlock(): poly(-1,nullptr){}
16
17      // inicijalizacija bloka polinomom
18      CrcBlock(const Polynomial& externPoly): poly(externPoly){}
19
20      // inicijalizacija bloka drugim blokom
21      CrcBlock(const CrcBlock& crc): poly(crc.poly){}
22
23
24      Polynomial getCrcGenerator() const;
25      // funkcija uzima Polynomial i kopira u član "poly"
26      void setCrcGenerator(const Polynomial&);
27      void setCrcGenerator(const CrcBlock& externCrc){poly = externCrc.poly;}
28
29      // funkcija uzima informacionu sekvencu (const int*) i njenu dužinu
30      // i na izlaz šalje kodnu sekvencu
31      int* encodeWord(int, const int*);
32      // funkcija uzima sekvencu bita (const int*) određene dužine (int)
33      // i vraća flag koji govori da li sekvenca sadrži grešku
34      int checksum(int, const int*);
35
36      // ispis crc bloka
37      void printCrcBlock() const;
38
39  };
40
41  #endif
```

CrcBlock.cpp

```
1  #include "CrcBlock.h"
2
3  Polynomial CrcBlock::getCrcGenerator() const
4  {
5      return poly;
6  }
7
8  void CrcBlock::setCrcGenerator(const Polynomial& externPoly)
9  {
10     int* intPtr;
11
12     intPtr = externPoly.getPolyCoefficients();
13     poly.setPolynomial(externPoly.getPolyDegree(),intPtr);
14
15     delete[] intPtr;
16 }
17
18 int* CrcBlock::encodeWord(int infoLength,const int* infoWord)
19 {
20     int codewordLength;
21     int* codeword;
22
23     codewordLength = infoLength+poly.getPolyDegree();
24
25     codeword = new int[codewordLength];
26
27     // informaciona reč se kopira na početak kodne reči
28     for(int m=0;m<infoLength;m++) codeword[m]=infoWord[m];
29
30     //informacioni frejm se dopunja nulama
31     for(int i=0;i<poly.getPolyDegree();i++)
32         codeword[infoLength+i]=0;
33
34     //kreira se novi objekat polinom koji predstavlja informacionu reč
35     Polynomial newPoly(codewordLength-1,codeword);
36
37     newPoly.calculateReminder(poly); // određuje se crc dodatak
38
39     int* intPtr;
40
41     intPtr = newPoly.getPolyCoefficients(); // vraća pokazivač na dinamički niz
42
43     for(int i=0;i<=newPoly.getPolyDegree();i++)
44         codeword[infoLength+i]=intPtr[i]; // crc dodatak se lepi na kraj kodne reči
45
46     delete[] intPtr;
47
48     return codeword;
49 }
50
```

```
51
52 int CrcBlock::checkSum(int codewordLength, const int codeword[])
53 {
54     int errorFlag = 0;
55
56     // objekat "newPoly" će se obrisati kad se završi izvršavanje funkcije,
57     // kada će se pozvati njegov destruktork koji će obrisati dinamičku memoriju
58     // dodeljenu objektu
59     Polynomial newPoly(codewordLength-1, codeword);
60     newPoly.calculateReminder(poly); // provera da li je crc dodatak ispravan
61
62     int* intPtr;
63
64     intPtr = newPoly.getPolyCoefficients();
65
66     for (int i=0; i<poly.getPolyDegree(); i++)
67     {
68         if(intPtr[i]!=0)
69         {
70             errorFlag = 1;
71             break;
72         }
73     }
74
75     delete[] intPtr;
76
77     return errorFlag;
78 }
79
80 void CrcBlock::printCrcBlock() const
81 {
82     cout << "CRC ";
83     poly.printPolynomial();
84 }
```

### CommBlock.h

```
1  #ifndef COMM_BLOCK_H
2      #define COMM_BLOCK_H
3
4      #include "Frame.h"
5      #include "Config.h"
6
7
8      class CommBlock
9      {
10
11      public:
12
13      CommBlock(const CrcBlock& crc, Config& simConfig, userSide externSide):
14          frame(crc, simConfig,externSide)
15      {
16          frame.setFrame(crc, simConfig,externSide);
17          simConfig.totalLength = simConfig.numFrames;
18      }
19
20      Frame frame;
21
22      };
23
24      #endif
```

### Frame.h

```
1  #ifndef FRAME_H
2      #define FRAME_H
3
4      #include "CrcBlock.h"
5      #include "Config.h"
6
7      class Frame
8      {
9          int numFrames;
10         int payloadSize;
11         int overheadSize;
12         int numDetectedFailures;
13         int* output;
14         int* outputCopy;
15
16         userSide side;
17         CrcBlock crc;
18
19         // frejmovi se proveravaju i svi frejmovi sa detektovanom
20         // greškom se brišu (dodeljuje im se vrednost 2)
21         void checkFrames(int*);
22     }
```

```

23     public:
24         // konstruktori
25         // inicijalizacija praznim blokom
26         Frame();
27         // kompletna inicijalizacija
28         Frame(const CrcBlock&,const Config&,userSide);
29         // destruktor
30         ~Frame() {delete[] output; delete[] outputCopy;};
31
32         void fillAndProcess(int*); // napuniti frejmove bitima za slanje
33         int* getOutput() const {return outputCopy;}
34         void setFrame(const CrcBlock&,const Config&,userSide);
35         // ispis podataka o frejmovima
36         void printFrameInfo() const;
37
38     };
39
40 #endif

```

### Frame.cpp

```

1  #include "Frame.h"
2
3
4  Frame::Frame():crc() //inicijalizacija praznim blokom
5  {
6      numFrames = 0;
7      payloadSize = 0;
8      overheadSize = 0;
9      numDetectedFailures = -1;
10     output = nullptr;
11     outputCopy = nullptr;
12 }
13
14
15 Frame::Frame(const CrcBlock& externCrc, const Config& myConfig, userSide externSide):
16     crc(externCrc) //kompletna inicijalizacija
17 {
18     numFrames = myConfig.numFrames;
19     payloadSize = myConfig.payloadSize;
20     numDetectedFailures = -1;
21
22     overheadSize = myConfig.overheadSize;
23
24     outputCopy = new int[numFrames*(payloadSize+overheadSize)];
25     output = new int[numFrames*(payloadSize+overheadSize)];
26
27     side = externSide;
28 }
29
30 void Frame::setFrame(const CrcBlock& externCrc,const Config& myConfig, userSide externSide)
31 {
32     numFrames = myConfig.numFrames;

```

```
33     payloadSize = myConfig.payloadSize;
34     numDetectedFailures = -1;
35
36     crc.setCrcGenerator(externCrc);
37
38     overheadSize = myConfig.overheadSize;
39
40     if(output != nullptr) {delete[] output; delete[] outputCopy;}
41
42     side = externSide;
43
44     if(side == transmit)
45     {
46         output = new int[numFrames*(payloadSize+overheadSize)];
47         outputCopy = new int[numFrames*(payloadSize+overheadSize)];
48     }
49     else
50     {
51         output = new int[numFrames*payloadSize];
52         outputCopy = new int[numFrames*payloadSize];
53     }
54 }
55
56
57 void Frame::fillAndProcess(int* inputStream)
58 {
59
60     if(side==transmit) // frejmovima se pre kopiranja u objektu dodaje CRC zaglavlje
61     {
62         int* tempInputFrame;
63         int* tempOutputFrame;
64
65         //tempOutputFrame = new int[payloadSize+overheadSize];
66
67         for(int i=0;i<numFrames;i++)
68         {
69
70             tempInputFrame = new int[payloadSize];
71
72             for(int j=0;j<payloadSize;j++)
73                 tempInputFrame[j]=inputStream[i*payloadSize+j];
74
75             tempOutputFrame = crc.encodeWord(payloadSize,tempInputFrame);
76             // kopiranje privremenog frejma u izlazni niz frejmova
77             for(int j=0;j<payloadSize+overheadSize;j++)
78                 output[i*(payloadSize+overheadSize)+j]=tempOutputFrame[j];
79
80             delete[] tempOutputFrame;
81             delete[] tempInputFrame;
82
83         }
84         // pravi se kopija izlaznog niza
85         for (int i=0;i<(payloadSize+overheadSize)*numFrames;i++)
86             outputCopy[i] = output[i];
87     }
```

```
88     else
89     {
90         checkFrames(inputStream); // vrši se provera CRC dodatka
91
92         int counter = 0;
93         for (int i=0;i<(payloadSize+overheadSize)*numFrames;i++)
94         {
95             if (i%(payloadSize+overheadSize)<payloadSize)
96                 output[counter++]=inputStream[i];
97         }
98         // pravi se kopija izlaznog niza
99         for (int i=0;i<payloadSize*numFrames;i++) outputCopy[i] = output[i];
100     }
101 }
102
103 // funkcija ne alokira novu memoriju (sva dinamički dodeljena
104 // memorija se briše u okviru funkcije)
105 void Frame::checkFrames(int* inputStream)
106 {
107     numDetectedFailures = 0;
108     int errorFlag;
109     int* tempFrame;
110
111     for (int i=0;i<numFrames;i++)
112     {
113         tempFrame = new int[payloadSize+overheadSize];
114
115         for (int j=0;j<payloadSize+overheadSize;j++)
116             tempFrame[j] = inputStream[i*(payloadSize+overheadSize)+j];
117
118         errorFlag = crc.checkSum(payloadSize+overheadSize,tempFrame);
119
120         delete[] tempFrame;
121
122         numDetectedFailures += errorFlag;
123
124         // obeležavanje pogrešno primljenih frejmova
125         // (detektovane greške se koduju numeričkom vrednošću "2")
126         if(errorFlag==1)
127         {
128             for (int j=0;j<payloadSize+overheadSize;j++)
129                 inputStream[i*(payloadSize+overheadSize)+j]=2;
130         }
131     }
132 }
133
134 void Frame::printFrameInfo() const
135 {
136     cout << "-----\n";
137     cout << "Num. of transmitted frames: " << numFrames << endl;
138     cout << "-----\n";
139     cout << "Payload size: " << payloadSize << endl;
140     cout << "-----\n";
141     cout << "Num. of detected failures: " << numDetectedFailures << endl;
142
143     crc.printCrcBlock();
144 }
```

SourceSink.h

```
1  #ifndef SOURCE_SINK_H
2  #define SOURCE_SINK_H
3
4  #include <stdlib.h>    /* srand, rand */
5  #include <time.h>      /* time */
6  #include <iostream>
7  #include "Config.h"
8
9  using namespace std;
10
11 class SourceSink
12 {
13     int sourceStreamSize;
14     double transProb01; // verovatnoće prelaza
15     double transProb10;
16     int* output; // izvor baferiše niz koji je poslao zbog vođenja statistike
17     int* outputCopy;
18
19     public:
20     // konstruktori
21     SourceSink() {sourceStreamSize = 0; transProb01 = 0.5; transProb10 = 0.5; output = nullptr;}
22     SourceSink(const Config& myConfig):
23         sourceStreamSize{myConfig.sourceStreamSize}, transProb01{ myConfig.transProb01 },
24         transProb10{myConfig.transProb10}
25     {
26         output = new int[sourceStreamSize];
27         outputCopy = new int[sourceStreamSize];
28     };
29
30     ~SourceSink() {delete[] output; delete[] outputCopy;} //briše se baferisani niz
31
32     void setSourceSize(int size)
33     {
34         sourceStreamSize = size;
35         output = new int[sourceStreamSize];
36         outputCopy = new int[sourceStreamSize];
37     };
38     void setTarnsProbs(double p01, double p10) {transProb01 = p01; transProb10 = p10;};
39
40     int* getOutput(){return outputCopy;}
41     void generateStream();
42
43     void calculateErrors(const Config&, int*);
44 };
45
46 #endif
```

SourceSink.cpp

```
1  #include "SourceSink.h"
2
3
4  void SourceSink::generateStream() // binarni izvor s memorijom prvog reda
5  {
6      double randomNumber;
7      int sourceState = 0;
8
9      // podesiti seed slučajnog generatora na osnovu trenutnog vremena;
10     // srand(const) omogućava ponovljivost niza slučajnog brojeva
11     srand(time(nullptr));
12
13     for (int i=0;i<sourceStreamSize;i++)
14     {
15         randomNumber = double(rand())/RAND_MAX;
16
17         switch (sourceState)
18         {
19             case 0:
20                 sourceState = 1-int(randomNumber>transProb01);
21                 output[i]=sourceState;
22                 break;
23
24             default:
25                 sourceState = 1-int(randomNumber<transProb10);
26                 output[i]=sourceState;
27             }
28         }
29     for (int i=0;i<sourceStreamSize;i++)
30         outputCopy[i] = output[i]; //kopiranje niza za izlaz
31 }
32
33 void SourceSink::calculateErrors(const Config& myConfig, int* receivedFrames)
34 {
35     int numUndetectedErrors = 0;
36
37     for (int i=0;i<myConfig.numFrames;i++)
38     {
39         if(receivedFrames[i*myConfig.payloadSize]!=2)
40         {
41             for (int j=0;j<myConfig.payloadSize;j++)
42             {
43                 if(receivedFrames[i*myConfig.payloadSize+j]!= output[i*myConfig.payloadSize+j])
44                 {
45                     numUndetectedErrors ++;
46                     break;
47                 }
48             }
49         }
50     }
51
52     cout << "Num. erroneous frames: " << numUndetectedErrors << endl;
53     cout << "-----\n";
54 }
55
```

### Channel.h

```
1  #ifndef CHANNEL_H
2  #define CHANNEL_H
3
4  #include <stdlib.h>    /* srand, rand */
5  #include <time.h>     /* time */
6
7  class Channel
8  {
9      double channelRate;
10     int channelSequenceLength;
11     int* output;
12     int* outputCopy;
13
14     void insertErrors();
15     //ulaz u kanal se baferiše
16     void fillChannel(int* sequence) {for(int i=0;i<channelSequenceLength;i++) output[i]=sequence[i];}
17
18     public:
19
20     Channel(int length,double rate=0):
21         channelRate(rate), channelSequenceLength(length)
22     {output = new int[channelSequenceLength]; outputCopy = new int[channelSequenceLength];};
23     ~Channel() {delete[] output; delete[] outputCopy;}
24     void fillAndProcess(int*);
25     void setChannelRate(double rate) {channelRate = rate;}
26     int* getOutput() {return outputCopy;}
27 };
28
29 #endif
```

### Channel.cpp

```
1  #include "Channel.h"
2
3  void Channel::insertErrors()
4  {
5
6      double randomNumber;
7      // podesiti seed slučajnog generatora na osnovu trenutnog vremena;
8      // srand(const) omogućava ponovljivost niza slučajnog brojeva
9      srand(time(nullptr));
10
11     for (int i=0;i<channelSequenceLength;i++)
12     {
13         randomNumber = double(rand())/RAND_MAX;
14
15         if(randomNumber<channelRate) output[i] = (output[i]+1)%2;
16     }
17 }
```

```

18
19
20 void Channel::fillAndProcess(int* sequence)
21 {
22     fillChannel(sequence); // baferiše se ulaz u kanal
23
24     insertErrors(); // dodaju se greške
25
26     // pravi se kopija za izlaz
27     for(int i=0;i<channelSequencLength;i++) outputCopy[i] = output[i];
28 }

```

Za analizu Gilbert-Eliotovog kanala, izmenjeni su sledeći fajlovi:

### Config.h

```

1  #ifndef CONFIG_H
2  #define CONFIG_H
3
4
5  // struktura za konfigurisanje simulacije
6
7  struct Config
8  {
9      //parametri izvora
10     const double transProb01 = 0.2;
11     const double transProb10 = 0.5;
12     const int sourceStreamSize = 4000000;
13     const int payloadSize = 40;
14     int numFrames = sourceStreamSize/payloadSize;
15
16     // parametri crc bloka
17     const int crcGenPoly[25] = { 1,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,1,0,0,0,1,1 };
18     const int overheadSize = sizeof(crcGenPoly)/sizeof(int)-1;
19     const int frameLength = numFrames*(payloadSize+overheadSize);
20     int totallength;
21     // verovatnoce gresaka
22
23     const double goodToBadErrorProb = 0.01; //Pd1 Td = 1/Pd1
24     const double badToGoodErrorProb = 0.01; //Pl1 Tl = 1/Pl1
25     const double averageErrorProb = 0.34; //Pe = (pd*Pl1 + pl*Pd1)/(Pl1 + Pd1)
26     const double errorProbArray[3] = { goodToBadErrorProb, badToGoodErrorProb, averageErrorProb };
27
28     const int numOfChannels = 1;
29 };
30
31 enum userSide{transmit, receive};
32
33 #endif

```

Channel.h

```

1  #ifndef CHANNEL_H
2  #define CHANNEL_H
3
4  #include <stdlib.h>      /* srand, rand */
5  #include <time.h>        /* time */
6  #include <iostream>
7
8  using namespace std;
9
10 class Channel
11 {
12     double goodToBadErrorProb;    // Pd1
13     double badToGoodErrorProb;    // Pl1
14     int goodChannelTime;          // Td
15     int badChannelTime;           // Tl
16     double goodChannelRate;       // pd
17     double badChannelRate;        // pl
18     int channelSequenceLength;
19     int* output;
20     int* outputCopy;
21
22     void insertErrors();
23     //ulaz u kanal se baferiše
24     void fillChannel(int* sequence)
25     {for(int i=0;i<channelSequenceLength;i++) output[i]=sequence[i];}
26
27     public:
28     Channel(int length, int goodChannelTime=0, int badChannelTime=0, double goodRate=0,
29             double badRate=0.5, double goodToBadErrorProb=0, double badToGoodErrorProb=0):
30         badChannelRate(badRate), channelSequenceLength(length)
31     {
32         output = new int[channelSequenceLength];
33         outputCopy = new int[channelSequenceLength];
34     };
35     ~Channel() {delete[] output; delete[] outputCopy;}
36     void fillAndProcess(int*);
37     void setChannelRate(const double* errorProbArray);
38     int* getOutput() {return outputCopy;}
39 };
40
41 #endif

```

Channel.cpp

```

1  #include "Channel.h"
2
3  void Channel::insertErrors()
4  {
5      double randomNumber1;
6      double randomNumber2;
7      double goodToGoodErrorProb = 1 - goodToBadErrorProb;
8      double badToBadErrorProb = 1 - badToGoodErrorProb;
9      int state = 0;

```

```
10 // podesiti seed slučajnog generatora na osnovu trenutnog vremena;
11 // srand(const) omogućava ponovljivost niza slučajnog brojeva
12 srand(time(nullptr));
13
14 for (int i = 0; i < channelSequenceLength; i++)
15 {
16     randomNumber1 = double(rand()) / RAND_MAX; // ovo je za stanje
17     randomNumber2 = double(rand()) / RAND_MAX; // ovo je za grešku
18     switch (state)
19     {
20     case 0: //good state
21         if (randomNumber2 < goodChannelRate) output[i] = (output[i] + 1) % 2;
22         if (randomNumber1 < goodToGoodErrorProb)
23             state = 0;
24         else
25             state = 1;
26         break;
27     default:
28         if (randomNumber2 < badChannelRate) output[i] = (output[i] + 1) % 2;
29         if (randomNumber1 < badToBadErrorProb)
30             state = 1;
31         else
32             state = 0;
33     }
34 }
35
36
37 void Channel::fillAndProcess(int* sequence)
38 {
39     fillChannel(sequence); // baferiše se ulaz u kanal
40
41     insertErrors(); // dodaju se greške
42
43     // pravi se kopija za izlaz
44     for(int i=0;i<channelSequenceLength;i++) outputCopy[i] = output[i];
45 }
46
47 void Channel::setChannelRate(const double* errorProbArray)
48 {
49     goodToBadErrorProb = errorProbArray[0]; //Pd1
50     badToGoodErrorProb = errorProbArray[1]; //Pld
51     double averageErrorProb = errorProbArray[2]; //Pe
52     double constRelation1 = goodToBadErrorProb / (badToGoodErrorProb + goodToBadErrorProb);
53     double constRelation2 = badToGoodErrorProb / (badToGoodErrorProb + goodToBadErrorProb);
54     goodChannelRate = abs(averageErrorProb - badChannelRate* constRelation1)/ constRelation2;
55     goodChannelTime = 1 / goodToBadErrorProb;
56     badChannelTime = 1 / badToGoodErrorProb;
57
58     cout << "Good state times: " << goodChannelTime << " | " << badChannelTime << endl;
59     cout << "-----\n";
60     cout << "Bad channel rate: " << badChannelRate << " | Good channel rate: " << goodChannelRate << endl;
61     cout << "-----\n";
62 }
```